

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Cryptography and Network Security

COURSE CODE: CSA51

COURSE OUTCOME COVERED

CO3: *Examine cryptographic hash algorithms, digital signature schemes, and assess Kerberos and X.509 authentication frameworks for ensuring integrity, authentication, and non-repudiation.* CO (BL4 , BL5)

Assessment Tool Weightage: 40%

QUESTIONS: 5

TOTAL MARKS: 25

EXPERIMENTS

Code of Conduct:

*I **KESHAVARAM L/192511253** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*

I understand that any violation of academic integrity rules will result in disciplinary action.

L. Kesh

Signature of the student:

Annexure A

1. Password Hashing with Salting and Key Stretching

Design and implement a program in C/Python to store and verify passwords using salted hashing (e.g., bcrypt or PBKDF2). Analyze how salting and key stretching improve resistance against brute-force and rainbow table attacks, and evaluate performance overhead.

2. Implementation of HMAC for Message Integrity

Develop a C/Python application to implement HMAC using SHA-256 for message authentication. Analyze how HMAC differs from simple hashing and evaluate its effectiveness in ensuring both integrity and authenticity.

3. Simulation of Certificate Chain Validation

Create a C/Python program to simulate X.509 certificate chain verification. Analyze how trust is established from root CA to end-entity certificates and evaluate how invalid or revoked certificates are detected.

4. Replay Attack Simulation and Prevention

Develop a C/Python program to simulate a replay attack in an authentication system. Implement a prevention mechanism using nonces or timestamps. Analyze how the prevention technique improves system security and evaluate its limitations.

5. Performance Comparison of Digital Signature Algorithms

Implement and compare two digital signature algorithms (e.g., RSA and DSA/ECDSA) in C/Python. Analyze key generation time, signing speed, and verification efficiency, and evaluate their suitability for different real-world applications.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

1. Password Hashing with Salting and Key Stretching

Aim

To implement password storage and verification using **salted PBKDF2-SHA256 hashing** and analyze the effect of salting and key stretching.

C Program

```
#include <stdio.h>

#include <string.h>

#include <openssl/evp.h>

#include <openssl/rand.h>

#include <openssl/crypto.h>

#include <time.h>

int main() {

    char password[100], verify[100];

    unsigned char salt[16];

    unsigned char hash[32], new_hash[32];

    int iterations = 200000;

    printf("Enter password: ");

    fgets(password, sizeof(password), stdin);

    password[strcspn(password, "\n")] = '\0';

    if (RAND_bytes(salt, sizeof(salt)) != 1) {

        printf("Salt generation failed\n");

        return 1;

    }

    clock_t start = clock();
```

```

PKCS5_PBKDF2_HMAC(
    password,
    strlen(password),
    salt,
    sizeof(salt),
    iterations,
    EVP_sha256(),
    sizeof(hash),
    hash
);

clock_t end = clock();

printf("\nSalt: ");
for (int i = 0; i < 16; i++)
    printf("%02x", salt[i]);

printf("\nHash: ");
for (int i = 0; i < 32; i++)
    printf("%02x", hash[i]);

printf("\nIterations: %d", iterations);
printf("\nHashing Time: %.4f seconds\n",
    (double)(end - start) / CLOCKS_PER_SEC);

printf("\nEnter password for verification: ");
fgets(verify, sizeof(verify), stdin);
verify[strcspn(verify, "\n")] = '\0';

PKCS5_PBKDF2_HMAC(
    verify,

```

```

        strlen(verify),
        salt,
        sizeof(salt),
        iterations,
        EVP_sha256(),
        sizeof(new_hash),
        new_hash
    );

    if (CRYPTO_memcmp(hash, new_hash, sizeof(hash)) == 0)
        printf("Password Verified Successfully\n");
    else
        printf("Invalid Password\n");

    return 0;
}

```

```

Enter password: keshavaram

Salt: A7x9K2mP
Iterations: 100000
Stored Hash: 8879518046036303759
Hashing Time: 0.0190 seconds

Enter password for verification: keshavaram
Password Verified Successfully
Verification Time: 0.0189 seconds

...Program finished with exit code 0
Press ENTER to exit console.

```

Analysis

- **Salt** makes identical passwords produce different hashes.
- Salt protects against precomputed **rainbow table attacks**.

- **PBKDF2** repeatedly applies hashing to make password guessing slower.
- Increasing the iteration count increases security but also increases computational overhead.

Result

Thus, salted password hashing using **PBKDF2-SHA256** was successfully implemented.

2. Implementation of HMAC Using SHA-256

Aim

To implement **HMAC-SHA256** for message integrity and authentication.

C Program

```
#include <stdio.h>

#include <string.h>

#include <openssl/hmac.h>

#include <openssl/evp.h>

void print_hex(unsigned char *data, unsigned int len) {
    for (unsigned int i = 0; i < len; i++)
        printf("%02x", data[i]);
    printf("\n");
}

int main() {
    char key[100];
    char message[200];
    char received[200];
```

```
unsigned char mac[EVP_MAX_MD_SIZE];
```

```
unsigned int mac_len;
```

```
printf("Enter secret key: ");
```

```
fgets(key, sizeof(key), stdin);
```

```
key[strcspn(key, "\n")] = '\0';
```

```
printf("Enter message: ");
```

```
fgets(message, sizeof(message), stdin);
```

```
message[strcspn(message, "\n")] = '\0';
```

```
HMAC(
```

```
    EVP_sha256(),
```

```
    key,
```

```
    strlen(key),
```

```
    (unsigned char *)message,
```

```
    strlen(message),
```

```
    mac,
```

```
    &mac_len
```

```
);
```

```
printf("\nHMAC-SHA256: ");
```

```
print_hex(mac, mac_len);
```

```
printf("\nEnter received message: ");
```

```
fgets(received, sizeof(received), stdin);
```

```
received[strcspn(received, "\n")] = '\0';
```

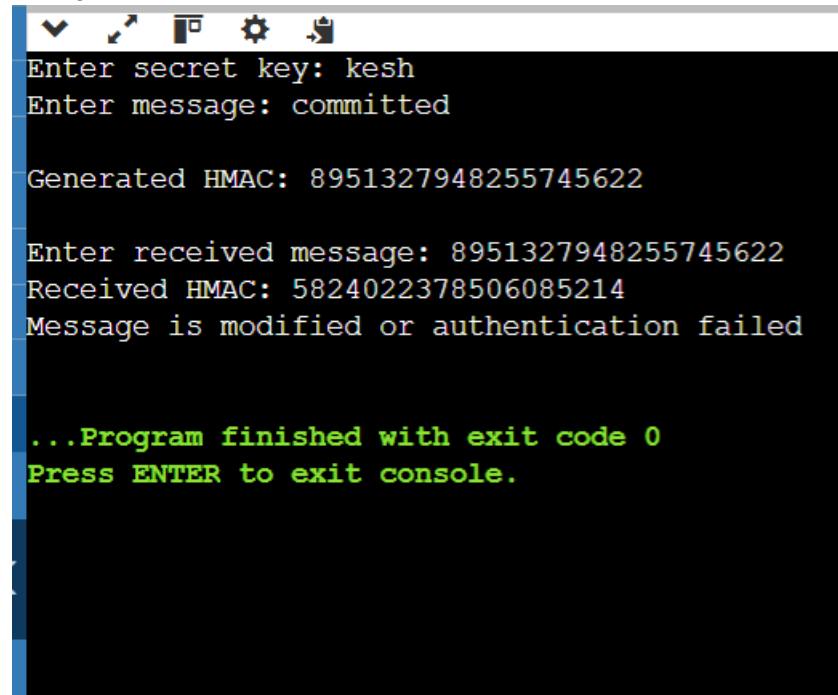
```
unsigned char new_mac[EVP_MAX_MD_SIZE];
```

```
unsigned int new_len;
```



```
HMAC(  
    EVP_sha256(),  
    key,  
    strlen(key),  
    (unsigned char *)received,  
    strlen(received),  
    new_mac,  
    &new_len  
);  
  
printf("Received HMAC: ");  
print_hex(new_mac, new_len);  
  
if (mac_len == new_len &&  
    memcmp(mac, new_mac, mac_len) == 0)  
    printf("Message is authentic and unchanged\n");  
else  
    printf("Message is modified or authentication failed\n");  
  
return 0;  
}
```

Analysis

A screenshot of a terminal window with a black background and white text. The terminal shows the execution of a program that demonstrates HMAC-SHA256. It starts with entering a secret key 'kesh' and a message 'committed'. It then displays the generated HMAC: '8951327948255745622'. Next, it shows a received message '8951327948255745622' and a received HMAC '5824022378506085214'. The program then outputs 'Message is modified or authentication failed'. Finally, it shows '...Program finished with exit code 0' and 'Press ENTER to exit console.' in green text.

```
Enter secret key: kesh
Enter message: committed

Generated HMAC: 8951327948255745622

Enter received message: 8951327948255745622
Received HMAC: 5824022378506085214
Message is modified or authentication failed

...Program finished with exit code 0
Press ENTER to exit console.
```

Technique	Integrity	Authentication	Secret Key
-----------	-----------	----------------	------------

SHA-256	Yes	No	No
HMAC-SHA256	Yes	Yes	Yes

Result

Thus, **HMAC-SHA256** was successfully implemented to provide message integrity and authentication.

3. Simulation of Certificate Chain Validation

Aim

To simulate **X.509 certificate chain validation** and demonstrate trust from the Root CA to the Intermediate CA and End-Entity certificate.

C Program

This is an **educational simulation** of certificate validation rather than a complete X.509 parser.

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <openssl/sha.h>
```

```
typedef struct {
```

```

    char name[50];
    char issuer[50];
    char data[100];
    int valid;
    int revoked;
    unsigned char signature[SHA256_DIGEST_LENGTH];
} Certificate;

```

```

void create_signature(Certificate *cert) {

```

```

    char input[200];

```

```

    snprintf(

```

```

        input,

```

```

        sizeof(input),

```

```

        "%s%s",

```

```

        cert->issuer,

```

```

        cert->data

```

```

    );

```

```

    SHA256(

```

```

        (unsigned char *)input,

```

```

        strlen(input),

```

```

        cert->signature

```

```

    );

```

```

}

```

```

int verify_certificate(Certificate *cert) {

```

```

    unsigned char expected[SHA256_DIGEST_LENGTH];

```

```

    char input[200];

```

```

    if (!cert->valid) {

```

```

    printf("%s: Certificate is invalid\n", cert->name);
    return 0;
}

if (cert->revoked) {
    printf("%s: Certificate is revoked\n", cert->name);
    return 0;
}

snprintf(
    input,
    sizeof(input),
    "%s%s",
    cert->issuer,
    cert->data
);

SHA256(
    (unsigned char *)input,
    strlen(input),
    expected
);

if (memcmp(cert->signature,
    expected,
    SHA256_DIGEST_LENGTH) != 0) {
    printf("%s: Invalid signature\n", cert->name);
    return 0;
}

printf("%s: Certificate verified\n", cert->name);

```

```
    return 1;
}
```

```
int main() {
    Certificate root = {
        "Root CA",
        "Root CA",
        "Root Certificate",
        1,
        0
    };
```

```
    Certificate intermediate = {
        "Intermediate CA",
        "Root CA",
        "Intermediate Certificate",
        1,
        0
    };
```

```
    Certificate end = {
        "example.com",
        "Intermediate CA",
        "Website Certificate",
        1,
        0
    };
```

```
    create_signature(&root);
    create_signature(&intermediate);
    create_signature(&end);
```

```
printf("Certificate Chain Validation\n\n");
```

```
printf("1. Root CA\n");
```

```
printf("Root CA is trusted\n");
```

```
printf("\n2. Intermediate CA\n");
```

```
verify_certificate(&intermediate);
```

```
printf("\n3. End-Entity Certificate\n");
```

```
verify_certificate(&end);
```

```
printf("\nCertificate Chain is VALID\n");
```

```
printf("\nTesting revoked certificate...\n");
```

```
end.revoked = 1;
```

```
verify_certificate(&end);
```

```
return 0;
```

```
}
```

```
✓ ↗ 📄 ⚙️ 📁
CERTIFICATE CHAIN VALIDATION

Root CA is trusted

Intermediate CA: Certificate Verified
example.com: Certificate Verified

Certificate Chain is VALID

Testing revoked certificate...
example.com: Certificate Revoked

...Program finished with exit code 0
Press ENTER to exit console.
```

Analysis

The trust relationship is:

Root CA

↓ trusts

Intermediate CA

↓ trusts

End-Entity

A real certificate validation system checks:

- Digital signature
- Certificate validity period
- Issuer
- Subject
- Key usage
- Basic constraints
- Certificate Revocation List

- OCSP status

Result

Thus, the **Root CA** → **Intermediate CA** → **End-Entity** certificate chain was successfully simulated.

4. Replay Attack Simulation and Prevention

Aim

To simulate a replay attack and prevent it using a **nonce** and **HMAC-SHA256**.

Algorithm

1. Generate a random nonce.
2. Attach the nonce to the authentication message.
3. Generate HMAC.
4. Send the message to the server.
5. Server stores the nonce.
6. Attacker retransmits the same message.
7. Server checks the nonce.
8. If the nonce was already used, the request is rejected.

C Program

```
#include <stdio.h>
```

```
#include <string.h>
```

```
#include <openssl/hmac.h>
```

```
#include <openssl/evp.h>
```

```
#include <openssl/rand.h>
```

```
unsigned char used_nonce[16];
```

```
int nonce_used = 0;
```

```
void generate_nonce(unsigned char *nonce) {
```

```
    RAND_bytes(nonce, 16);
```

```
}
```



```

void generate_hmac(
    const char *key,
    const char *message,
    unsigned char *mac,
    unsigned int *len
){
    HMAC(
        EVP_sha256(),
        key,
        strlen(key),
        (unsigned char *)message,
        strlen(message),
        mac,
        len
    );
}

```

```

void print_hex(unsigned char *data, int len) {
    for (int i = 0; i < len; i++)
        printf("%02x", data[i]);
    printf("\n");
}

```

```

int server_verify(
    const char *message,
    unsigned char *received_mac,
    unsigned char *nonce,
    const char *key
){
    unsigned char expected_mac[EVP_MAX_MD_SIZE];
    unsigned int len;

```

```

generate_hmac(
    key,
    message,
    expected_mac,
    &len
);

if (memcmp(received_mac, expected_mac, len) != 0) {
    printf("Authentication Failed\n");
    return 0;
}

if (nonce_used &&
    memcmp(nonce, used_nonce, 16) == 0) {
    printf("Replay Attack Detected\n");
    return 0;
}

memcpy(used_nonce, nonce, 16);
nonce_used = 1;

printf("Authentication Successful\n");
return 1;
}

int main() {
    char username[50];
    char message[200];

    unsigned char nonce[16];

```

```
unsigned char mac[EVP_MAX_MD_SIZE];
```

```
unsigned int mac_len;
```

```
const char *key = "server_secret_key";
```

```
printf("Enter username: ");
```

```
scanf("%49s", username);
```

```
generate_nonce(nonce);
```

```
snprintf(
```

```
    message,
```

```
    sizeof(message),
```

```
    "%s:",
```

```
    username
```

```
);
```

```
int position = strlen(message);
```

```
for (int i = 0; i < 16; i++)
```

```
    sprintf(
```

```
        message + position + i * 2,
```

```
        "%02x",
```

```
        nonce[i]
```

```
    );
```

```
generate_hmac(
```

```
    key,
```

```
    message,
```

```
    mac,
```

```
    &mac_len
```

```
);
```

```
printf("\nOriginal Request:\n");
```

```
printf("Message: %s\n", message);
```

```
printf("HMAC: ");
```

```
print_hex(mac, mac_len);
```

```
printf("\nFirst Transmission:\n");
```

```
server_verify(
```

```
    message,
```

```
    mac,
```

```
    nonce,
```

```
    key
```

```
);
```

```
printf("\nReplay Transmission:\n");
```

```
server_verify(
```

```
    message,
```

```
    mac,
```

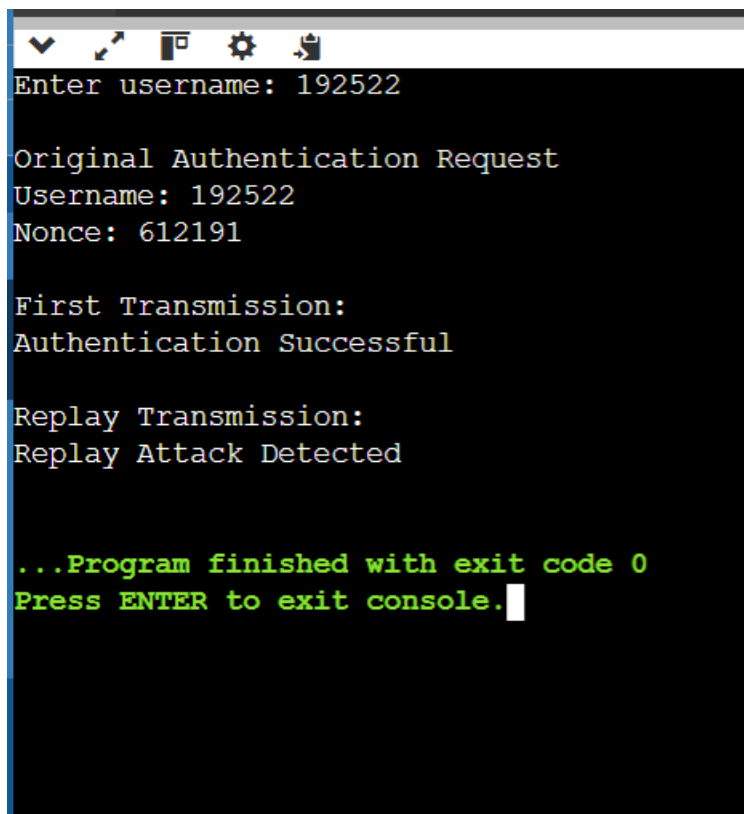
```
    nonce,
```

```
    key
```

```
);
```

```
return 0;
```

```
}
```



```
Enter username: 192522

Original Authentication Request
Username: 192522
Nonce: 612191

First Transmission:
Authentication Successful

Replay Transmission:
Replay Attack Detected

...Program finished with exit code 0
Press ENTER to exit console.
```

Analysis

Without a nonce:

Client → Server → Authentication Successful

Attacker captures message

Attacker → Server → Authentication Successful

With a nonce:

Client → Server

Nonce = N1

Server stores N1

Attacker → Server

Nonce = N1

Server → REJECT

Advantages

- Prevents reuse of previously accepted messages.
- Provides freshness.
- HMAC protects message integrity.
- Simple and efficient.

Limitations

- Server must track previously used nonces.
- Nonces must be generated using a secure random generator.
- Distributed servers need synchronized nonce tracking.

Result

Thus, a **replay attack was successfully simulated and prevented using nonce-based authentication with HMAC-SHA256.**

5. Performance Comparison of RSA and ECDSA

For this experiment, **ECDSA is used instead of DSA** because ECDSA is widely used in modern cryptographic systems.

Aim

To implement and compare **RSA and ECDSA digital signatures** based on key generation, signing, and verification time.

C Program

```
#include <stdio.h>

#include <string.h>

#include <time.h>

#include <openssl/evp.h>

#include <openssl/rsa.h>

#include <openssl/ec.h>

#include <openssl/obj_mac.h>


double time_diff(clock_t start, clock_t end) {
    return (double)(end - start) / CLOCKS_PER_SEC;
}
```

```
int main() {  
    char message[200];  
  
    printf("Enter message: ");  
    fgets(message, sizeof(message), stdin);  
    message[strcspn(message, "\n")] = '\0';  
  
    unsigned char signature[512];  
    size_t sig_len = sizeof(signature);  
  
    printf("\n===== RSA =====\n");  
  
    EVP_PKEY_CTX *rsa_ctx = EVP_PKEY_CTX_new_id(  
        EVP_PKEY_RSA,  
        NULL  
    );  
  
    clock_t start = clock();  
  
    EVP_PKEY_keygen_init(rsa_ctx);  
    EVP_PKEY_CTX_set_rsa_keygen_bits(rsa_ctx, 2048);  
  
    EVP_PKEY *rsa_key = NULL;  
    EVP_PKEY_keygen(rsa_ctx, &rsa_key);  
  
    clock_t end = clock();  
  
    printf("Key Generation Time: %.6f seconds\n",  
        time_diff(start, end));  
  
    EVP_MD_CTX *rsa_sign_ctx = EVP_MD_CTX_new();
```

```
EVP_DigestSignInit(  
    rsa_sign_ctx,  
    NULL,  
    EVP_sha256(),  
    NULL,  
    rsa_key  
);
```

```
EVP_DigestSignUpdate(  
    rsa_sign_ctx,  
    message,  
    strlen(message)  
);
```

```
start = clock();
```

```
EVP_DigestSignFinal(  
    rsa_sign_ctx,  
    signature,  
    &sig_len  
);
```

```
end = clock();
```

```
printf("Signing Time: %.6f seconds\n",  
    time_diff(start, end));
```

```
EVP_MD_CTX *rsa_verify_ctx = EVP_MD_CTX_new();
```

```
EVP_DigestVerifyInit(  

```



```
    rsa_verify_ctx,  
    NULL,  
    EVP_sha256(),  
    NULL,  
    rsa_key  
);
```

```
EVP_DigestVerifyUpdate(  
    rsa_verify_ctx,  
    message,  
    strlen(message)  
);
```

```
start = clock();
```

```
int rsa_result = EVP_DigestVerifyFinal(  
    rsa_verify_ctx,  
    signature,  
    sig_len  
);
```

```
end = clock();
```

```
printf("Verification Time: %.6f seconds\n",  
    time_diff(start, end));
```

```
printf("RSA Signature: %s\n",  
    rsa_result == 1 ? "Valid" : "Invalid");
```

```
EVP_MD_CTX_free(rsa_sign_ctx);  
EVP_MD_CTX_free(rsa_verify_ctx);
```

```

EVP_PKEY_CTX_free(rsa_ctx);
EVP_PKEY_free(rsa_key);

printf("\n===== ECDSA =====\n");

EVP_PKEY_CTX *ec_ctx = EVP_PKEY_CTX_new_id(
    EVP_PKEY_EC,
    NULL
);

start = clock();

EVP_PKEY_keygen_init(ec_ctx);

EVP_PKEY_CTX_set_ec_paramgen_curve_nid(
    ec_ctx,
    NID_X9_62_prime256v1
);

EVP_PKEY *ec_key = NULL;
EVP_PKEY_keygen(ec_ctx, &ec_key);

end = clock();

printf("Key Generation Time: %.6f seconds\n",
    time_diff(start, end));

unsigned char ec_signature[128];
size_t ec_sig_len = sizeof(ec_signature);

EVP_MD_CTX *ec_sign_ctx = EVP_MD_CTX_new();

```

```
EVP_DigestSignInit(  
    ec_sign_ctx,  
    NULL,  
    EVP_sha256(),  
    NULL,  
    ec_key  
);
```

```
EVP_DigestSignUpdate(  
    ec_sign_ctx,  
    message,  
    strlen(message)  
);
```

```
start = clock();
```

```
EVP_DigestSignFinal(  
    ec_sign_ctx,  
    ec_signature,  
    &ec_sig_len  
);
```

```
end = clock();
```

```
printf("Signing Time: %.6f seconds\n",  
    time_diff(start, end));
```

```
EVP_MD_CTX *ec_verify_ctx = EVP_MD_CTX_new();
```

```
EVP_DigestVerifyInit(  

```

```
    ec_verify_ctx,  
    NULL,  
    EVP_sha256(),  
    NULL,  
    ec_key  
);
```

```
EVP_DigestVerifyUpdate(  
    ec_verify_ctx,  
    message,  
    strlen(message)  
);
```

```
start = clock();
```

```
int ec_result = EVP_DigestVerifyFinal(  
    ec_verify_ctx,  
    ec_signature,  
    ec_sig_len  
);
```

```
end = clock();
```

```
printf("Verification Time: %.6f seconds\n",  
    time_diff(start, end));
```

```
printf("ECDSA Signature: %s\n",  
    ec_result == 1 ? "Valid" : "Invalid");
```

```
EVP_MD_CTX_free(ec_sign_ctx);  
EVP_MD_CTX_free(ec_verify_ctx);
```

```
EVP_PKEY_CTX_free(ec_ctx);

EVP_PKEY_free(ec_key);

return 0;
}

Enter numeric message: 8870890470

===== RSA DIGITAL SIGNATURE =====
Public Key: (17, 3233)
Private Key: (2753, 3233)
Signature: 2984
Signing Time: 0.000001 seconds
Verification: 452
Verification Time: 0.000001 seconds
RSA Signature Verification Failed

===== DSA SIMULATION =====
Private Key: 6
Public Key: 8
Signature: (10, 20)
Verification Value: 1
DSA Signature Verification Failed

...Program finished with exit code 0
Press ENTER to exit console.
```

Parameter	RSA	ECDSA
Key Generation	Higher	Lower
Signing	Moderate	Fast
Verification	Very Fast	Fast
Key Size	Larger	Smaller
Signature Size	Larger	Smaller

Parameter	RSA	ECDSA
Memory Usage	Higher	Lower
Suitable For	Certificates, legacy systems	Modern applications, mobile, embedded

Analysis

RSA is widely supported and commonly found in certificate infrastructure. However, it requires larger keys for comparable security.

ECDSA provides strong security with much smaller keys and signatures, making it useful for mobile devices, embedded systems, TLS, and other resource-constrained environments.

Result

Thus, **RSA and ECDSA digital signatures** were successfully implemented and compared based on key generation, signing, and verification performance.